

```
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/loan_prediction.csv')

# Display the first 5 rows of the DataFrame
display(df.head())
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amo
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	

```
# Display general information about the DataFrame, including data types and non-null values
display(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID                614 non-null    object
1   Gender                  601 non-null    object
2   Married                  611 non-null    object
3   Dependents              599 non-null    object
4   Education               614 non-null    object
5   Self_Employed           582 non-null    object
6   ApplicantIncome         614 non-null    int64
7   CoapplicantIncome       614 non-null    float64
8   LoanAmount              592 non-null    float64
9   Loan_Amount_Term        600 non-null    float64
10  Credit_History           564 non-null    float64
11  Property_Area           614 non-null    object
12  Loan_Status             614 non-null    object
dtypes: float64(4), int64(1), object(8)
memory usage: 67.2+ KB
None
```

```
# Calculate the percentage of missing values for each column
missing_values = df.isnull().sum()
missing_percentage = (missing_values / len(df)) * 100

# Display columns with missing values and their percentages, sorted in descending order
missing_info = pd.DataFrame({
    'Missing Count': missing_values,
    'Missing Percentage': missing_percentage
})
missing_info = missing_info[missing_info['Missing Count'] > 0].sort_values(by='Missing Percentage', ascending=False)

display(missing_info)
```

	Missing Count	Missing Percentage	
Credit_History	50	8.143322	
Self_Employed	32	5.211726	
LoanAmount	22	3.583062	
Dependents	15	2.442997	
Loan_Amount_Term	14	2.280130	
Gender	13	2.117264	
Married	3	0.488599	

Next steps:

[Generate code with missing_info](#)[New interactive sheet](#)

```
# Impute missing values

# Categorical columns: fill with mode
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
df['Married'] = df['Married'].fillna(df['Married'].mode()[0])
df['Dependents'] = df['Dependents'].fillna(df['Dependents'].mode()[0])
df['Self_Employed'] = df['Self_Employed'].fillna(df['Self_Employed'].mode()[0])
df['Credit_History'] = df['Credit_History'].fillna(df['Credit_History'].mode()[0])
df['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mode()[0])

# Numerical columns: fill with median (robust to outliers)
df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].median())

# Verify that there are no more missing values
print('Missing values after imputation:')
display(df.isnull().sum())
```

Missing values after imputation:

	0
Loan_ID	0
Gender	0
Married	0
Dependents	0
Education	0
Self_Employed	0
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	0
Loan_Amount_Term	0
Credit_History	0
Property_Area	0
Loan_Status	0

dtype: int64

```
# Identify categorical columns (object type)
categorical_cols = df.select_dtypes(include='object').columns

print('Unique values and their counts for categorical columns:')
for col in categorical_cols:
    print(f"\nColumn: {col}")
    display(df[col].value_counts())
```


Unique values and their counts for categorical columns:

Column: Loan_ID

count

Loan_ID	
LP002990	1
LP001002	1
LP001003	1
LP001005	1
LP001006	1
...	...
LP001024	1
LP001020	1
LP001018	1
LP001014	1
LP001013	1

614 rows × 1 columns

dtype: int64

Column: Gender

```
# Drop Loan_ID column as it is just an identifier
df = df.drop('Loan_ID', axis=1)

# Convert '3+' in Dependents to '3'
df['Dependents'] = df['Dependents'].replace('3+', '3').astype(int)

# Map Loan_Status target variable to numerical (Y=1, N=0)
df['Loan_Status'] = df['Loan_Status'].map({'Y': 1, 'N': 0})

# Identify categorical columns for one-hot encoding (excluding Loan_Status and Dependents which are already handled)
categorical_cols_for_ohe = ['Gender', 'Married', 'Education', 'Self_Employed', 'Property_Area']

# Apply one-hot encoding
df = pd.get_dummies(df, columns=categorical_cols_for_ohe, drop_first=True)

print("DataFrame after dropping Loan_ID, handling Dependents, mapping Loan_Status, and One-Hot Encoding:")
display(df.head())
display(df.info())
```

Column: Dependents

count

Dependents	
0	360
1	102
2	101
3+	51

dtype: int64

Column: Education

count

Education	
Graduate	480
Not Graduate	134

dtype: int64

Column: Self_Employed

```

Dataframe after dropping Loan_ID, handling Dependents, mapping Loan_Status, and One-Hot Encoding:
Self_Employed
Dependents  ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  Credit_History  Loan_Status  Gender_Male  Married_Yes
0           0                5849                0.0        128.0             360.0             1.0             1             True
1           1                4583             1508.0        128.0             360.0             1.0             0             True
2           0                3000                0.0         66.0             360.0             1.0             1             True
3           0             2583             2358.0        120.0             360.0             1.0             1             True
4           0                6000                0.0       141.0             360.0             1.0             1             True

dtype: int64
Column: Property_Area
count
0           4
1           4
2           4
3           4
4           4
dtype: int64

<class 'pandas.core.frame.DataFrame'>
Index: Int64Index: 230 to 613
Data columns (total 13 columns):
#   Urban  Non-Null Count  Dtype
---  -
0   Rural  179              614 non-null  int64
1   ApplicantIncome  614 non-null  int64
2   CoapplicantIncome  614 non-null  float64
3   LoanAmount  614 non-null  float64
4   Loan_Amount_Term  614 non-null  float64
5   Credit_History  614 non-null  float64
6   Loan_Status  614 non-null  int64
7   Gender_Male  614 non-null  bool
8   Married_Yes  614 non-null  bool
9   Education_Not_Graduate  614 non-null  bool
10  Self_Employed_Yes  614 non-null  bool
11  Property_Area_Semiurban  614 non-null  bool
12  Property_Area_Urban  614 non-null  bool
dtypes: bool(6), float64(4), int64(3)
memory usage: 42.0 KB
None
```

```

# Check the distribution of the target variable
print("Distribution of Loan_Status:")
display(y.value_counts())
print("\nPercentage of Loan_Status:")
display(y.value_counts(normalize=True) * 100)
```

Distribution of Loan_Status:

count	
Loan_Status	
1	422
0	192

dtype: int64

Percentage of Loan_Status:

proportion	
Loan_Status	
1	68.729642
0	31.270358

dtype: float64

```

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

print("\nDistribution of y_train before SMOTE:")
display(y_train.value_counts(normalize=True) * 100)
```

```

Shape of X_train: (491, 12)
Shape of X_test: (123, 12)
Shape of y_train: (491,)
Shape of y_test: (123,)

```

Distribution of y_train before SMOTE:

	proportion
Loan_Status	
1	68.635438
0	31.364562

dtype: float64

```

from imblearn.over_sampling import SMOTE

# Apply SMOTE to the training data
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print("Shape of X_train after SMOTE:", X_train_smote.shape)
print("Shape of y_train after SMOTE:", y_train_smote.shape)

print("\nDistribution of y_train after SMOTE:")
display(y_train_smote.value_counts(normalize=True) * 100)

```

```

Shape of X_train after SMOTE: (674, 12)
Shape of y_train after SMOTE: (674,)

```

Distribution of y_train after SMOTE:

	proportion
Loan_Status	
1	50.0
0	50.0

dtype: float64

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix

# Initialize and train the Logistic Regression model
log_reg_model = LogisticRegression(random_state=42, solver='liblinear') # liblinear is good for small datasets
log_reg_model.fit(X_train_smote, y_train_smote)

# Make predictions on the test set
y_pred_log_reg = log_reg_model.predict(X_test)
y_proba_log_reg = log_reg_model.predict_proba(X_test)[:, 1]

# Evaluate the Logistic Regression model
print("Logistic Regression Model Evaluation:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_log_reg):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_log_reg):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_log_reg):.4f}")
print(f"F1-Score: {f1_score(y_test, y_pred_log_reg):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_proba_log_reg):.4f}")

print("\nConfusion Matrix:")
display(pd.DataFrame(confusion_matrix(y_test, y_pred_log_reg), index=['Actual N', 'Actual Y'], columns=['Predicted N', 'Pre

```

Logistic Regression Model Evaluation:
 Accuracy: 0.8130
 Precision: 0.8370
 Recall: 0.9059
 F1-Score: 0.8701
 ROC-AUC: 0.8220

Confusion Matrix:

	Predicted N	Predicted Y
Actual N	23	15
Actual Y	8	77

```
from sklearn.tree import DecisionTreeClassifier
```

```
# Initialize and train the Decision Tree Classifier model
dec_tree_model = DecisionTreeClassifier(random_state=42)
dec_tree_model.fit(X_train_smote, y_train_smote)
```

```
# Make predictions on the test set
y_pred_dec_tree = dec_tree_model.predict(X_test)
y_proba_dec_tree = dec_tree_model.predict_proba(X_test)[: , 1]
```

```
# Evaluate the Decision Tree model
print("Decision Tree Classifier Model Evaluation:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_dec_tree):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_dec_tree):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_dec_tree):.4f}")
print(f"F1-Score: {f1_score(y_test, y_pred_dec_tree):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_proba_dec_tree):.4f}")
```

```
print("\nConfusion Matrix:")
display(pd.DataFrame(confusion_matrix(y_test, y_pred_dec_tree), index=['Actual N', 'Actual Y'], columns=['Predicted N', 'Pr
```

Decision Tree Classifier Model Evaluation:
 Accuracy: 0.6748
 Precision: 0.8169
 Recall: 0.6824
 F1-Score: 0.7436
 ROC-AUC: 0.6701

Confusion Matrix:

	Predicted N	Predicted Y
Actual N	25	13
Actual Y	27	58

```
from sklearn.ensemble import RandomForestClassifier
```

```
# Initialize and train the Random Forest Classifier model
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train_smote, y_train_smote)
```

```
# Make predictions on the test set
y_pred_rf = rf_model.predict(X_test)
y_proba_rf = rf_model.predict_proba(X_test)[: , 1]
```

```
# Evaluate the Random Forest model
print("Random Forest Classifier Model Evaluation:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_rf):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_rf):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_rf):.4f}")
print(f"F1-Score: {f1_score(y_test, y_pred_rf):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_proba_rf):.4f}")
```

```
print("\nConfusion Matrix:")
display(pd.DataFrame(confusion_matrix(y_test, y_pred_rf), index=['Actual N', 'Actual Y'], columns=['Predicted N', 'Predicted Y']
```

Random Forest Classifier Model Evaluation:
Accuracy: 0.8293
Precision: 0.8556
Recall: 0.9059
F1-Score: 0.8800
ROC-AUC: 0.7565

Confusion Matrix:

	Predicted N	Predicted Y
Actual N	25	13
Actual Y	8	77

```
# Store and compare model performance
model_performance = {
    'Logistic Regression': {
        'Accuracy': accuracy_score(y_test, y_pred_log_reg),
        'Precision': precision_score(y_test, y_pred_log_reg),
        'Recall': recall_score(y_test, y_pred_log_reg),
        'F1-Score': f1_score(y_test, y_pred_log_reg),
        'ROC-AUC': roc_auc_score(y_test, y_proba_log_reg)
    },
    'Decision Tree': {
        'Accuracy': accuracy_score(y_test, y_pred_dec_tree),
        'Precision': precision_score(y_test, y_pred_dec_tree),
        'Recall': recall_score(y_test, y_pred_dec_tree),
        'F1-Score': f1_score(y_test, y_pred_dec_tree),
        'ROC-AUC': roc_auc_score(y_test, y_proba_dec_tree)
    },
    'Random Forest': {
        'Accuracy': accuracy_score(y_test, y_pred_rf),
        'Precision': precision_score(y_test, y_pred_rf),
        'Recall': recall_score(y_test, y_pred_rf),
        'F1-Score': f1_score(y_test, y_pred_rf),
        'ROC-AUC': roc_auc_score(y_test, y_proba_rf)
    }
}
```

```
performance_df = pd.DataFrame(model_performance).T.round(4)
print("\nModel Performance Comparison:")
display(performance_df)
```

Model Performance Comparison:

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8130	0.8370	0.9059	0.8701	0.8220
Decision Tree	0.6748	0.8169	0.6824	0.7436	0.6701
Random Forest	0.8293	0.8556	0.9059	0.8800	0.7565

Next steps: [Generate code with performance_df](#) [New interactive sheet](#)

```
import matplotlib.pyplot as plt
from sklearn.metrics import RocCurveDisplay

# Create a figure and an axes object for the plot
fig, ax = plt.subplots(figsize=(10, 8))

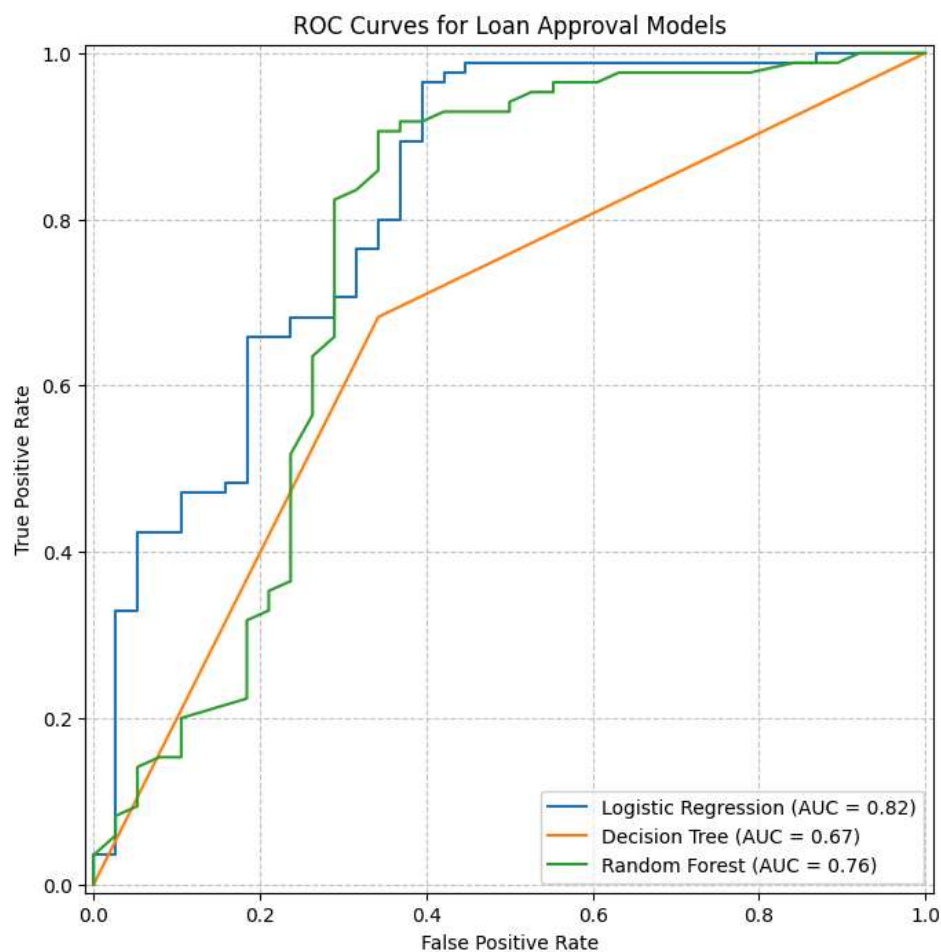
# Plot ROC curve for Logistic Regression
RocCurveDisplay.from_estimator(log_reg_model, X_test, y_test, ax=ax, name='Logistic Regression')

# Plot ROC curve for Decision Tree
RocCurveDisplay.from_estimator(dec_tree_model, X_test, y_test, ax=ax, name='Decision Tree')

# Plot ROC curve for Random Forest
RocCurveDisplay.from_estimator(rf_model, X_test, y_test, ax=ax, name='Random Forest')

# Add title and labels
ax.set_title('ROC Curves for Loan Approval Models')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
```

```
ax.grid(linestyle='--', alpha=0.7)
plt.show()
```



```
from sklearn.metrics import PrecisionRecallDisplay

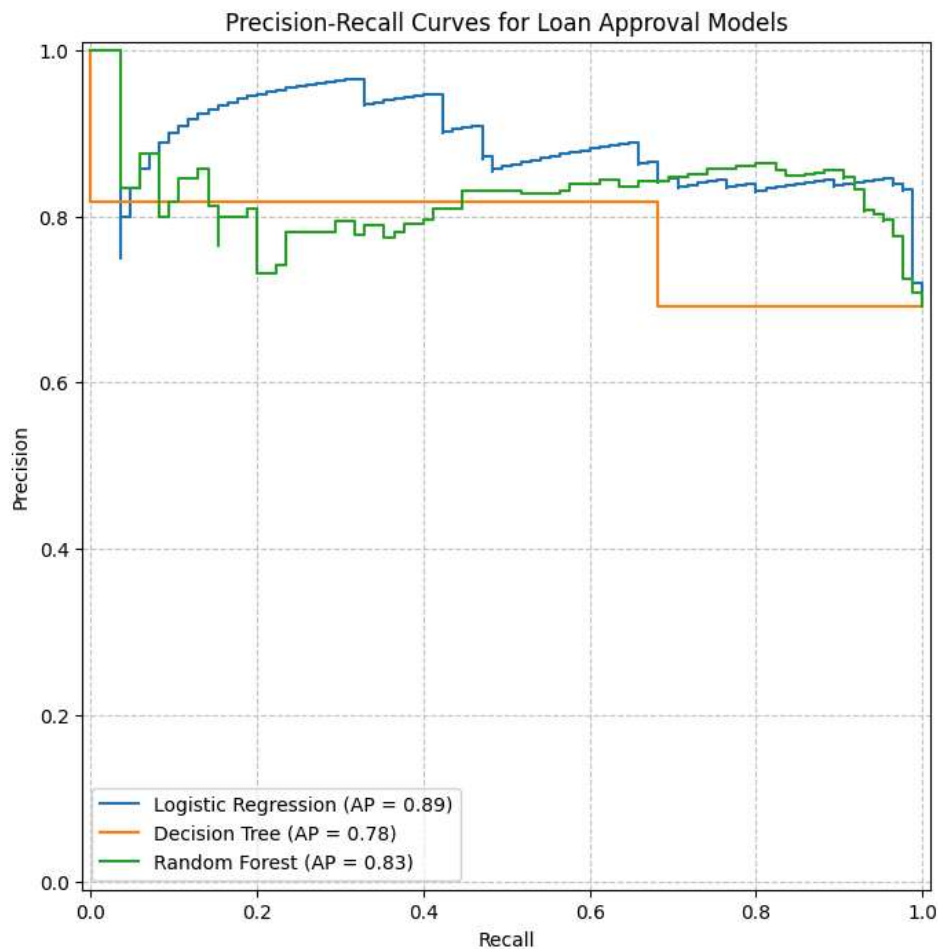
# Create a figure and an axes object for the plot
fig_pr, ax_pr = plt.subplots(figsize=(10, 8))

# Plot Precision-Recall curve for Logistic Regression
PrecisionRecallDisplay.from_estimator(log_reg_model, X_test, y_test, ax=ax_pr, name='Logistic Regression')

# Plot Precision-Recall curve for Decision Tree
PrecisionRecallDisplay.from_estimator(dec_tree_model, X_test, y_test, ax=ax_pr, name='Decision Tree')

# Plot Precision-Recall curve for Random Forest
PrecisionRecallDisplay.from_estimator(rf_model, X_test, y_test, ax=ax_pr, name='Random Forest')

# Add title and labels
ax_pr.set_title('Precision-Recall Curves for Loan Approval Models')
ax_pr.set_xlabel('Recall')
ax_pr.set_ylabel('Precision')
ax_pr.grid(linestyle='--', alpha=0.7)
plt.show()
```



Confusion Matrices

```
# Logistic Regression Confusion Matrix
print("Logistic Regression Confusion Matrix:")
display(pd.DataFrame(confusion_matrix(y_test, y_pred_log_reg), index=['Actual N', 'Actual Y'], columns=['Predicted N', 'Predicted Y']))

# Decision Tree Confusion Matrix
print("\nDecision Tree Confusion Matrix:")
display(pd.DataFrame(confusion_matrix(y_test, y_pred_dec_tree), index=['Actual N', 'Actual Y'], columns=['Predicted N', 'Predicted Y']))

# Random Forest Confusion Matrix
print("\nRandom Forest Confusion Matrix:")
display(pd.DataFrame(confusion_matrix(y_test, y_pred_rf), index=['Actual N', 'Actual Y'], columns=['Predicted N', 'Predicted Y']))
```

Logistic Regression Confusion Matrix:

	Predicted N	Predicted Y
Actual N	23	15
Actual Y	8	77

Decision Tree Confusion Matrix:

	Predicted N	Predicted Y
Actual N	25	13
Actual Y	27	58

Random Forest Confusion Matrix:

	Predicted N	Predicted Y
Actual N	25	13
Actual Y	8	77

Feature Importance for Random Forest Model

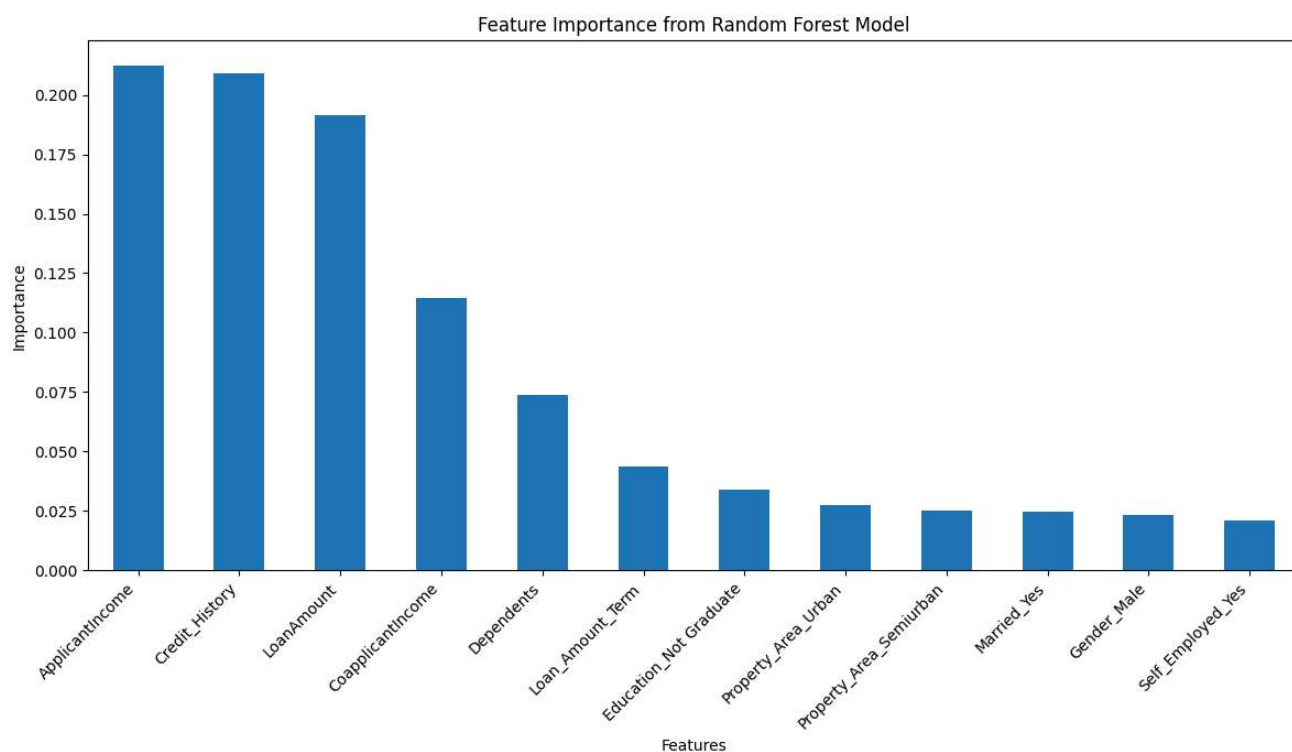
```
import matplotlib.pyplot as plt

# Get feature importances from the Random Forest model
feature_importances = rf_model.feature_importances_

# Get feature names from X (original features before scaling, or scaled features if using X directly)
# Assuming X_train_smote columns represent the features after preprocessing
feature_names = X_train_smote.columns

# Create a pandas Series for easier sorting and plotting
importance_df = pd.Series(feature_importances, index=feature_names).sort_values(ascending=False)

# Plot feature importances
plt.figure(figsize=(12, 7))
importance_df.plot(kind='bar')
plt.title('Feature Importance from Random Forest Model')
plt.xlabel('Features')
plt.ylabel('Importance')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Cost-Benefit Analysis: Threshold Tuning for Random Forest

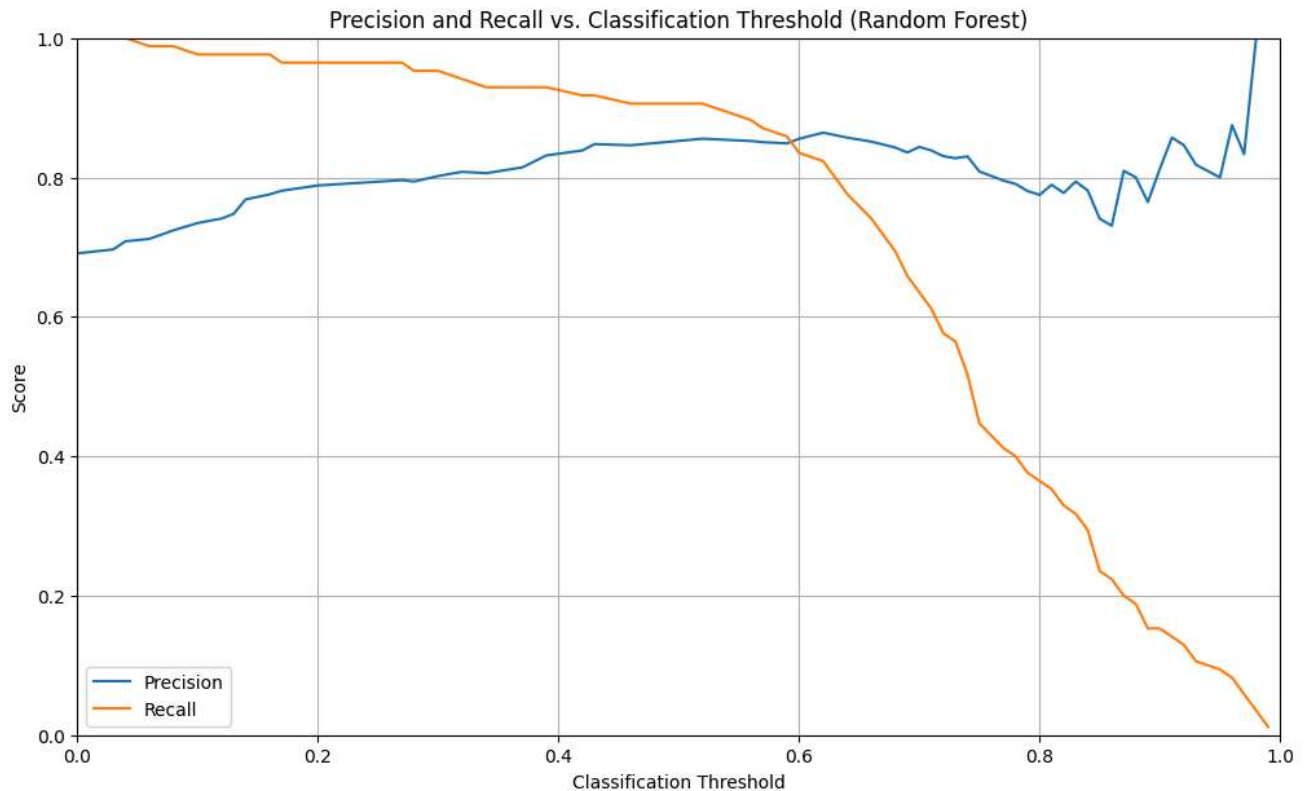
```
from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt

# Get predicted probabilities for the positive class from the Random Forest model
y_proba = rf_model.predict_proba(X_test)[: , 1]

# Calculate precision, recall, and thresholds
```

```
precision, recall, thresholds = precision_recall_curve(y_test, y_proba)

# Plot the precision-recall-threshold curve
plt.figure(figsize=(12, 7))
plt.plot(thresholds, precision[:-1], label='Precision')
plt.plot(thresholds, recall[:-1], label='Recall')
plt.xlabel('Classification Threshold')
plt.ylabel('Score')
plt.title('Precision and Recall vs. Classification Threshold (Random Forest)')
plt.legend()
plt.grid(True)
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.show()
```



The plot above illustrates the trade-off between Precision and Recall at various classification thresholds for the Random Forest model.

- **Higher Thresholds:** Increase precision (fewer false positives), but decrease recall (more false negatives).
- **Lower Thresholds:** Increase recall (fewer false negatives), but decrease precision (more false positives).

To conduct a thorough cost-benefit analysis:

1. **Quantify Costs:** Assign monetary costs to false positives (e.g., cost of a defaulting loan) and false negatives (e.g., lost revenue from a missed approved loan).
2. **Quantify Benefits:** Assign monetary benefits to true positives (e.g., profit from a successfully repaid loan) and true negatives.
3. **Simulate Outcomes:** For each threshold, use the confusion matrix (or predicted counts) to calculate the total cost/benefit based on your defined monetary values.
4. **Select Optimal Threshold:** Choose the threshold that maximizes net profit or minimizes overall cost, according to your business objectives.

This plot serves as a crucial visual aid in making informed decisions about the optimal operating point for your model in a real-world scenario.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Separate features (X) and target (y)
```

```
X = df.drop('Loan_Status', axis=1)
y = df['Loan_Status']

# Identify numerical columns for scaling (excluding boolean columns)
numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns

# Apply StandardScaler to numerical columns
scaler = StandardScaler()
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])

print("DataFrame after Feature Scaling:")
display(X.head())
display(y.head())
```

DataFrame after Feature Scaling:

	Dependents	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Gender_Male	Married_Yes	Loan_Status
0	-0.737806	0.072991	-0.554487	-0.211241	0.273231	0.411733	True	False	1
1	0.253470	-0.134412	-0.038732	-0.211241	0.273231	0.411733	True	True	0
2	-0.737806	-0.393747	-0.554487	-0.948996	0.273231	0.411733	True	True	1
3	-0.737806	-0.462062	0.251980	-0.306435	0.273231	0.411733	True	True	1
4	-0.737806	0.097728	-0.554487	-0.056551	0.273231	0.411733	True	False	1

dtype: int64

Business Interpretation and Model Trade-offs

In the context of loan approval, different metrics hold varying importance depending on the business objective.

- **Precision:** The ability of the model not to label as positive a sample that is negative (i.e., minimizing false positives). In loan approval, high precision means fewer approved loans to applicants who will default (minimizing financial losses for the bank).
- **Recall:** The ability of the model to find all the positive samples (i.e., minimizing false negatives). High recall means approving a larger percentage of eligible applicants (maximizing business growth and inclusivity).
- **F1-Score:** The harmonic mean of precision and recall. It's a good measure when you need to balance both precision and recall.
- **ROC-AUC:** Represents the ability of the model to distinguish between positive and negative classes. A higher AUC indicates a better overall model performance regardless of the classification threshold.

Model Comparison:

1. **Logistic Regression:**
 - **Pros:** Good balance of precision and recall, decent ROC-AUC. It's interpretable, which is valuable for regulated industries like finance. Its Recall (0.9059) is high, meaning it's good at identifying actual loan defaulters. It has the highest ROC-AUC among the three, suggesting good discrimination ability.
 - **Cons:** Accuracy is slightly lower than Random Forest.
2. **Decision Tree:**
 - **Pros:** Easy to understand and visualize. It has a high Precision (0.8169) which is good to avoid false positives (approving bad loans).
 - **Cons:** Significantly lower Accuracy, Recall, F1-Score, and ROC-AUC compared to the other two models. It is prone to overfitting if not properly pruned.
3. **Random Forest:**

- **Pros:** Highest Accuracy (0.8293) and F1-Score (0.8800), and tied for highest Recall (0.9059). Generally robust against overfitting. Very good at identifying actual loan defaulters. Its Precision is also good.
- **Cons:** Lower ROC-AUC than Logistic Regression. Less interpretable than Logistic Regression or a single Decision Tree.

Trade-offs and Recommendation:

- **If minimizing financial losses from defaults is the absolute priority (high cost of false positives):** A model with higher precision would be preferred. Random Forest has a slightly higher precision than Logistic Regression. However, Logistic Regression's overall balance and interpretability make it a strong contender.
- **If maximizing the number of approved eligible loans is crucial (high cost of false negatives):** Both Logistic Regression and Random Forest excel with very high Recall scores (0.9059). Both models are good at identifying applicants who *should* be approved.

Considering the performance metrics, **Random Forest** appears to be the best performing model overall in terms of Accuracy, F1-Score, and Recall. While Logistic Regression has a slightly better ROC-AUC and is more interpretable, Random Forest's superior F1-Score and Accuracy suggest better overall predictive power. For a business context, Random Forest would likely lead to a better balance of identifying good loan applicants while maintaining reasonable control over potential defaults.

Suggested Deployment Threshold:

The default classification threshold for most models is 0.5. This means if the predicted probability of loan approval is > 0.5 , the loan is approved. However, this threshold can be adjusted based on business risk appetite.

- **Current Threshold (0.5):** The model performance metrics discussed above are based on this default threshold.
- **Adjusting the Threshold:**
 - **Increase threshold (>0.5):** This would make the model more conservative, leading to fewer approved loans but also fewer defaults (higher precision, lower recall). This might be desired if the bank wants to significantly reduce risk.
 - **Decrease threshold (<0.5):** This would make the model more lenient, leading to more approved loans but potentially more defaults (lower precision, higher recall). This might be considered if the bank is looking for aggressive growth and is willing to accept a higher risk.

Recommendation: Given that Random Forest has the highest F1-Score and a high Recall, indicating a good balance, we could start with the default threshold of **0.5**. However, it is crucial to perform a **cost-benefit analysis** specific to the bank's operational costs, potential losses from defaults, and profits from approved loans. This analysis would involve plotting Precision-Recall curves or ROC curves to visualize the trade-offs at different thresholds and selecting the threshold that optimizes business outcomes.

Conclusion and Report Summary

This notebook demonstrates the development of a supervised machine learning model to predict loan approval based on borrower features. The process involved several key stages:

1. **Data Loading and Initial Inspection:** The `loan_prediction.csv` dataset was loaded, and its structure, data types, and missing values were initially examined.